DragonflyDB Architecture Overview & 6.43M QPS

Dissection: Arpit Bhayani & Roman Gershman
Focus: Multicore Scaling & Forkless Snapshots

1. Executive Summary & The 6.43M QPS Milestone

In modern database engineering, the industry consensus has long maintained that in-memory key-value stores should scale horizontally via multi-node clustering. However, cloud infrastructure economics have dramatically shifted: single virtual instances now provide 64 to 128 physical CPU cores, gigabytes of CPU cache, and terabytes of RAM on unified motherboards.

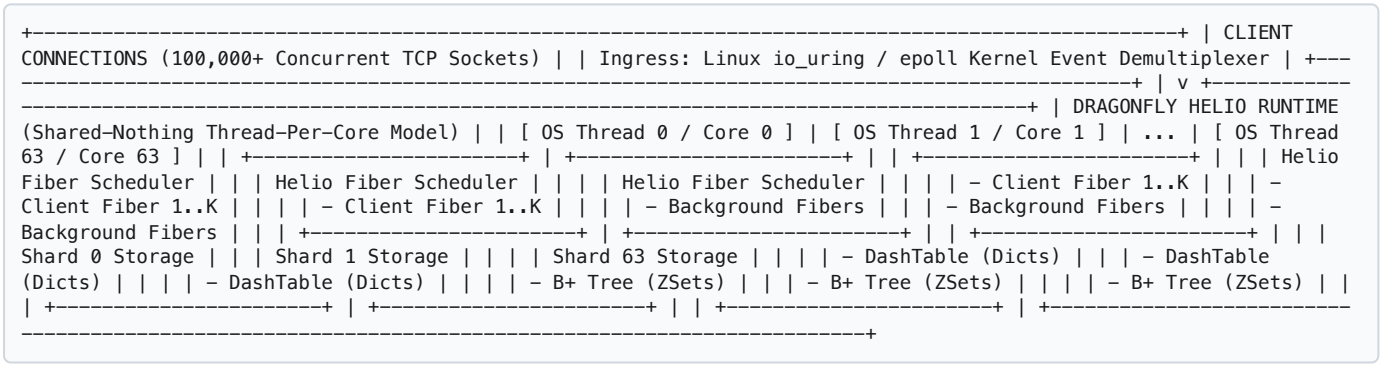
> **The Benchmark:** On an AWS `c7gn.16xlarge` instance equipped with 64 ARM-based AWS Graviton3 Neoverse-V1 cores, **DragonflyDB achieved 6.43 million operations per second** with sub-millisecond P99 tail latency. This represents a **25x throughput improvement over Redis** and a **4x improvement over KeyDB**, while utilizing $30\% - 40\%$ less memory.

2. Why Redis Hits a Ceiling on Modern Multicore Hardware

Redis was architected in 2009 around a single-threaded event loop. While elegant for small machines, this design creates crippling bottlenecks in multicore environments:

- **Severe Core Starvation:** On a 64-core server, Redis utilizes only 1 core ($ pprox 1.5\%$), leaving the remaining 63 cores completely idle while maxing out at $ pprox 150 ext{k QPS}$.
- **Cluster Overhead & Operational Friction:** Harnessing a 64-core instance with Redis requires running 64 separate node processes in Redis Cluster mode, introducing cluster bus gossip noise, proxy layers, and fatal `CROSSSLOT` multi-key transaction errors.
- **Copy-on-Write (COW) Memory Doubling:** When persisting data via RDB snapshots, Redis invokes the Linux `fork()` system call. Under heavy write traffic, page fault overhead and Copy-on-Write can double memory consumption ($100 ext{ GB} o 200 ext{ GB}$), frequently triggering the Linux Out-Of-Memory (OOM) killer.

3. DragonflyDB Architectural Blueprint



4. The Helio Engine: Cooperative Fibers

DragonflyDB is built on **Helio**, Roman Gershman's custom C++ asynchronous runtime library. Helio replaces OS-level threads (such as `pthreads`) with **cooperative fibers**:

- **Lightweight Green Threads:** Fibers allocate tiny, segmented stacks ($ pprox 16 ext{ KB}$ vs $2 ext{ MB} - 8 ext{ MB}$ default OS thread stacks). Millions of concurrent client connections can be maintained with minimal RAM overhead.
- **Sub-50ns Context Switching:** Fiber context switches occur completely in user space via register swaps. There is zero Linux kernel transition, zero TLB shootdowns, and zero OS scheduling jitter.
- **Non-Blocking Cooperative Scheduling:** When a fiber waits for socket network I/O or multi-shard transaction synchronization, it yields execution. The CPU core instantly switches to another ready client fiber, maintaining 100% core saturation.
- **Asynchronous I/O via io_uring:** Helio natively leverages Linux `io_uring` to submit and complete batched network and storage operations with a single kernel transition.

5. Shared-Nothing Thread-per-Core Sharding

Unlike KeyDB, which added multithreading to Redis by wrapping a global hash table in spinlocks (causing massive CPU cache ping-pong and lock convoys), DragonflyDB enforces strict **shared-nothing memory isolation**:

- **Shard Partitioning:** The global keyspace is hash-partitioned into N shards matching the number of CPU cores. Thread T_i exclusively mutates Shard S_i.
- **Zero Mutation Locks:** Single-key queries ($\gt 95\%$ of production workloads) execute without mutexes, spinlocks, or atomic CAS instructions.
- **Lock-Free Inter-Thread Messaging:** When cross-shard communication is required, threads exchange asynchronous messages across lock-free ring buffers (SPSC queues), avoiding memory contention.

6. Advanced In-Memory Data Structures

> **Modernizing Redis Legacy Types:** Redis was designed prior to modern CPU cache optimizations and relies heavily on pointer-heavy heap structures that induce CPU stall cycles.

- **DashTable for Key Dictionaries:** Replaces Redis `dictEntry` separate chaining with cache-line-aligned (64B) segments, 1-byte fingerprints, SIMD vector probing, and stash overflow. Cuts metadata overhead by $30\% - 40\%$ and eliminates global rehashing stalls.
- **B+ Trees for Sorted Sets:** Redis implements `ZSET` via a SkipList. SkipLists allocate separate heap nodes linked by probabilistic pointers, causing poor cache locality. Dragonfly implements Sorted Sets with compact **B+ Trees**, packing multiple keys and scores into contiguous nodes, slashing pointer overhead by 60% and optimizing range scans.

Forkless Snapshots, Engine Comparisons & Verdict

7. Forkless Snapshotting: Ending the Copy-on-Write Tax

In Redis, generating an RDB snapshot relies on the Linux `fork()` system call. The OS duplicates memory pages via Copy-on-Write (COW) whenever a key is written during the backup. On active write systems, this doubles RAM usage, causes latency spikes, and leads to kernel OOM panics.

DragonflyDB implements a **Forkless Snapshotting Engine**:

- **Bucket-Level User-Space Isolation:** Instead of relying on coarse $4 ext{ KB}$ OS memory pages, Dragonfly tracks snapshot state at fine-grained **bucket granularity** within DashTables.
- **User-Space Pre-Mutation Serialization:** When a client attempts to mutate an item in a bucket that has not yet been serialized, the shard fiber serializes the pre-mutation bucket state to a memory buffer before applying the write.
- **Direct I/O Disk Streaming:** A dedicated `SaveBody` fiber streams serialized chunks directly to disk using `O_DIRECT`, completely bypassing the OS page cache and preventing memory buffer bloating.

8. Engine Comparison: Redis vs KeyDB vs DragonflyDB

Architectural Attribute	Redis 7.x (Standalone)	KeyDB (Multi-Threaded)	DragonflyDB
Concurrency Model	Single-threaded event loop	Shared memory + spinlocks	Shared-nothing thread-per-core (Helio fibers)
64-Core Throughput	$ pprox 150 ext{k QPS}$ (1 core used)	$ pprox 1.5 ext{M QPS}$ (lock convoy)	**6.43 Million QPS (Linear multicore scale)**
Dictionary Structure	Chained hash table (`dictEntry`)	Chained table with locks	**DashTable (Segmented, 64B cacheline aligned)**
Sorted Set Engine	SkipList + Hash Table	SkipList + Hash Table	**Contiguous B+ Trees**
Snapshotting Mechanism	OS `fork()` (COW memory spikes)	OS `fork()` (COW spikes)	**Forkless bucket streaming (Zero COW spikes)**
Memory Overhead	High ($24 ext{B}+$ metadata tax)	High (metadata + locks)	**$30\% - 40\%$ physical RAM reduction**

9. First-Principles Summary & Design Lessons

- **Hardware Co-Design Is Essential:** Maximum throughput is achieved not by adding more distributed nodes, but by aligning software architecture with hardware primitives: 64-byte cache lines, SIMD registers, and non-blocking fiber runtimes.
- **Shared-Nothing Beats Shared-Memory:** Lock-free thread-per-core architectures eliminate cache coherency traffic and spinlock convoys, delivering predictable linear vertical scaling.
- **Replace Kernel Primitives with Application Logic:** Relying on kernel primitives like `fork()` for consistency introduces uncontrolled memory blowups. Implementing fine-grained user-space isolation guarantees rock-solid memory stability.